



University
of Exeter

COMM113

Deep Learning – Week 10

Transformer and Language Models II

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

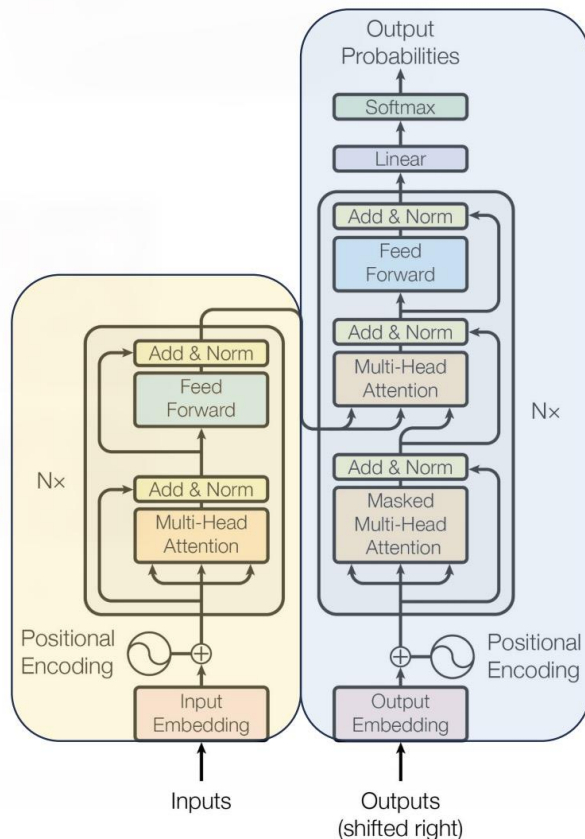
Review - Transformers

Input – input tokens
Output – hidden states

Model can see all timesteps

**Does not usually output tokens, so
no inherent auto-regressivity**

Representation



Input – output tokens and hidden states*

Output – output tokens

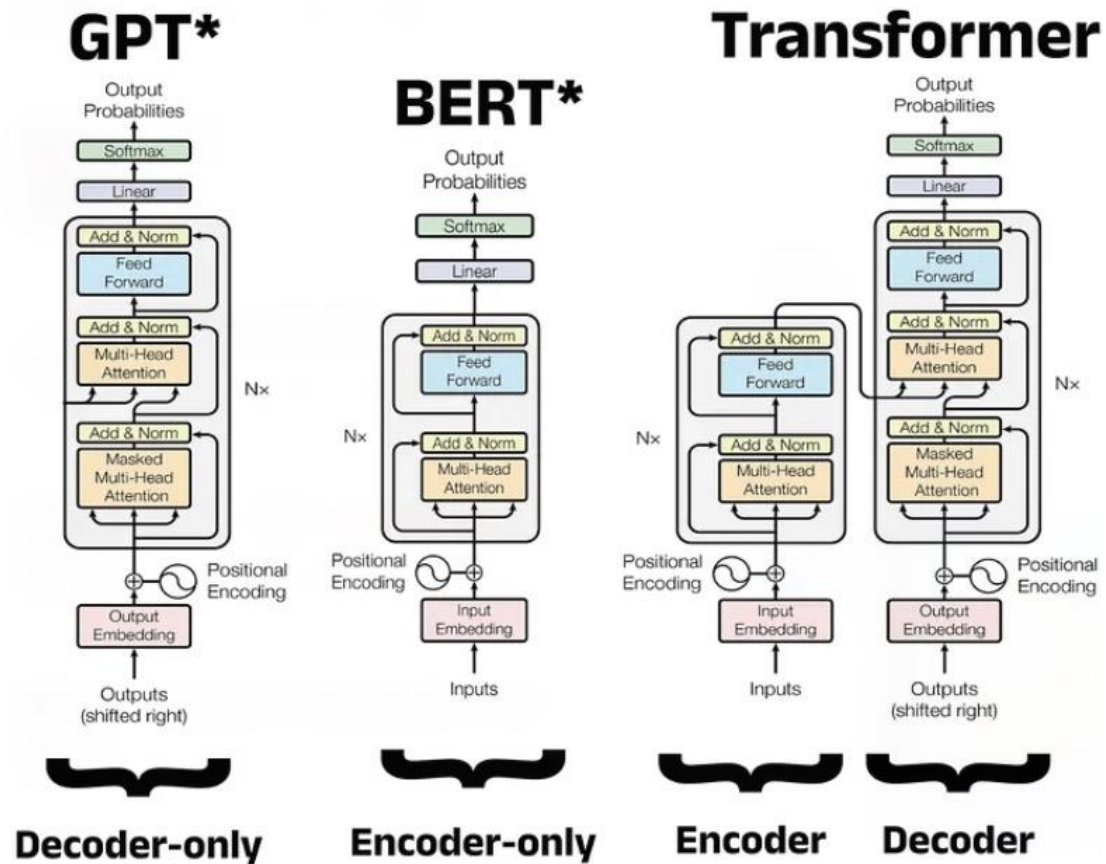
**Model can only see previous
timesteps**

**Model is auto-regressive with
previous timesteps' outputs**

Generation

Review

GPT predicts the next word based only on the words before it. This is why GPT is excellent at text generation.



BERT sees the entire sentence at once. It **understands** context in both directions.

How LLMs Actually Work – Training, Prompting, and Inference

Overview

- How LLMs are trained step-by-step
- What happens during inference
- Why prompting changes behaviour

How powerful are the current large models?

[Sora 2](#)

[GPT5](#)

LLM Training

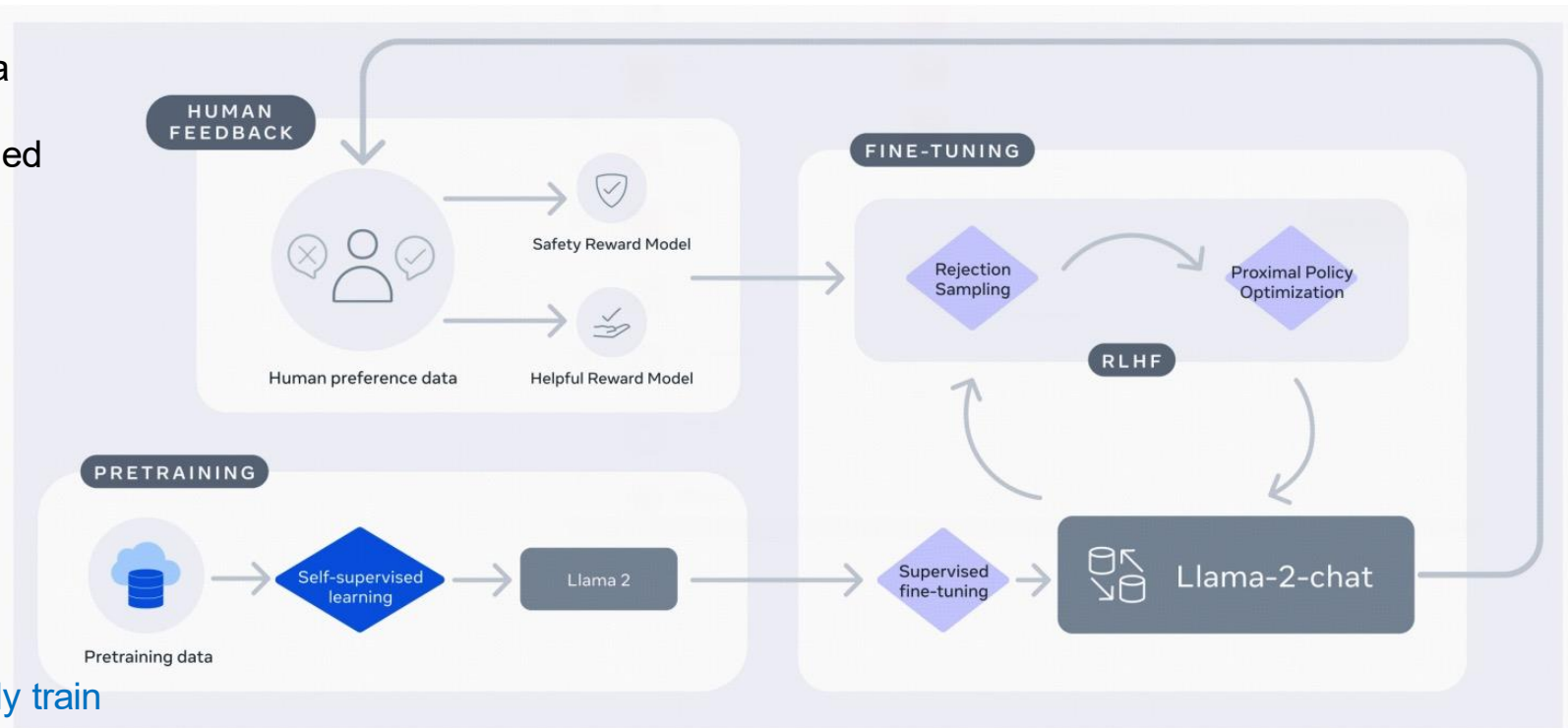
Training LLMs - Llama as an example



Pretraining =
massive raw data

SFT = task-focused
correction

RLHF = human
alignment



Q: Why not directly train
with human feedback?

Pre-training -> Supervised Fine-tuning (SFT) -> RLHF Pre-training -> Post-training
Pre-training -> Mid-training -> Post-training

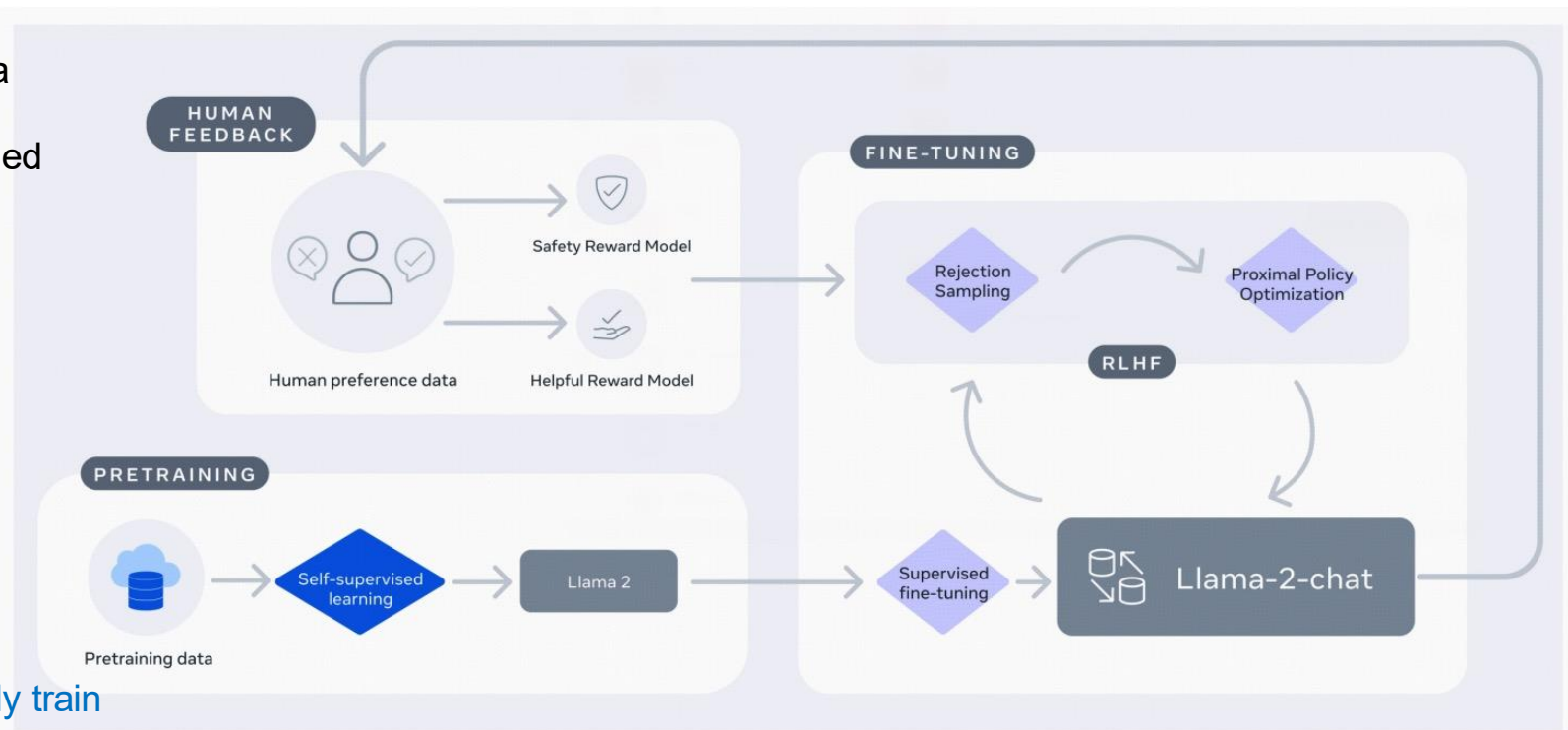
Training LLMs - Llama as an example



Pretraining =
massive raw data

SFT = task-focused
correction

RLHF = human
alignment



Q: Why not directly train
with human feedback?

A: Too expensive and
limited in scale.

Pre-training -> Supervised Fine-tuning (SFT) -> RLHF Pre-training -> Post-training
Pre-training -> Mid-training -> Post-training

Why is it called pre-training?

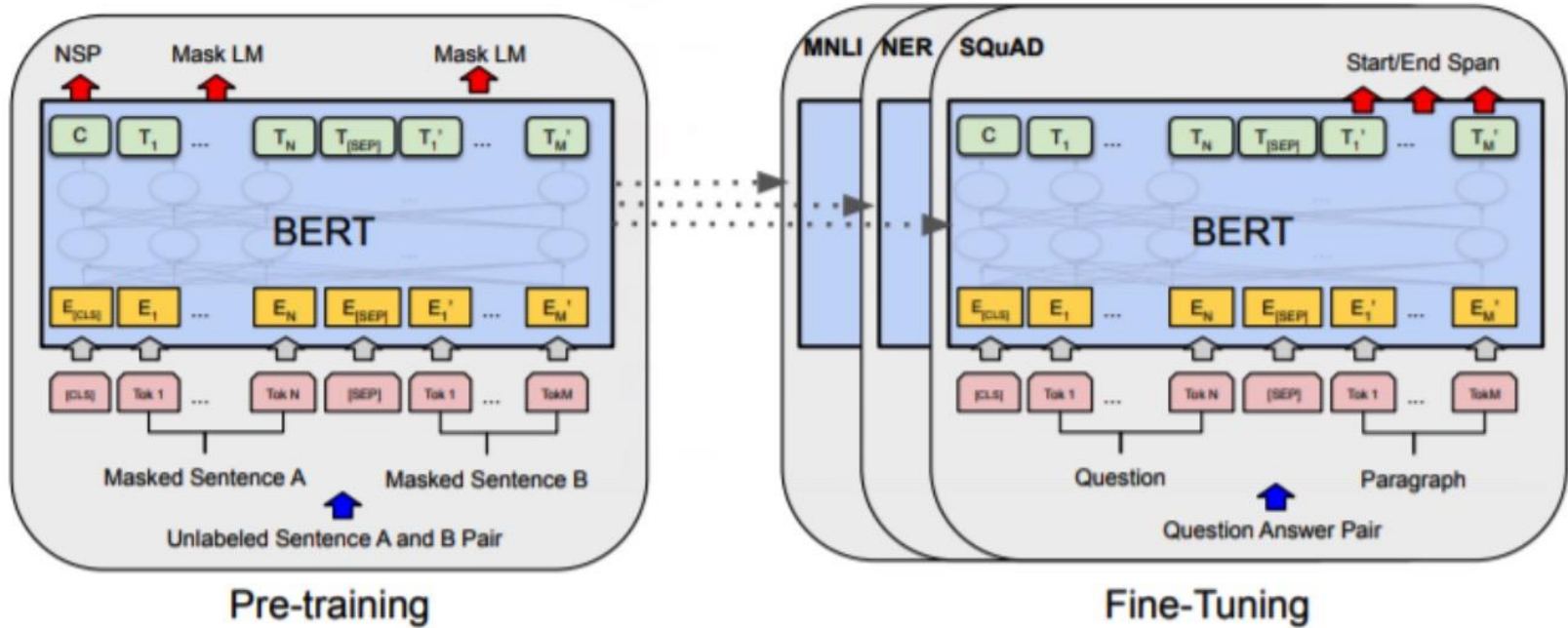


Image Credit: BERT Paper

“Pre-”training happens before training (fine-tuning)!

like learning vocabulary before writing papers

Pre-training Data

Common
Crawl

Colossal Clean
Crawled Corpus (C4)



WIKIPEDIA
The Free Encyclopedia



arXiv

StackExchange

Example: Llama1 Pre-training Data Mixture

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

1.4 Trillion Tokens!

How Large are 1T Tokens?

Physical Size (if printed)

- Average words per page: A typical page contains about 300-500 words.
- Words from 1 trillion tokens: Assuming 750 billion words, and an average of 400 words per page.
- Total Pages: Approximately **1.875 billion pages**.

How Large are 1T Tokens?

Physical Size (if printed)

- Average words per page: A typical page contains about 300-500 words.
- Words from 1 trillion tokens: Assuming 750 billion words, and an average of 400 words per page.
- Total Pages: Approximately **1.875 billion pages**.

Digital Storage

- Character Encoding: Assuming each character takes up 1 byte (in a simple encoding like ASCII), 1 trillion tokens (4 trillion characters) would require about **4 terabytes (TB) of storage**.

How Large are 1T Tokens?

Physical Size (if printed)

- Average words per page: A typical page contains about 300-500 words.
- Words from 1 trillion tokens: Assuming 750 billion words, and an average of 400 words per page.
- Total Pages: Approximately **1.875 billion pages**.

Digital Storage

- Character Encoding: Assuming each character takes up 1 byte (in a simple encoding like ASCII), 1 trillion tokens (4 trillion characters) would require about **4 terabytes (TB) of storage**.

Reading Time

- Reading Speed: The average reading speed is about 200-250 words per minute.
- Time to Read 750 Billion Words: At 200 words per minute, it would take about 3.75 billion minutes, or approximately **7,125 years of continuous reading**.

Tokeniser

“We tokenize the data with the **bytepair encoding (BPE)** algorithm (Sennrich et al., 2015), using the implementation from SentencePiece (Kudo and Richardson, 2018). Notably, we split all numbers into individual digits, and fallback to bytes to decompose unknown UTF-8 characters.” (—from Llama1 Paper)

- Incrementally combine together the most frequent token pairs

```
{'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
```

```
pairs = get_stats(vocab)
```

```
[(('e', 's'), 9), (('s', 't'), 9), (('t', '</w>'), 9), (('w', 'e'), 8), (('l', 'o'), 7), ...]
```

```
vocab = merge_vocab(pairs[0], vocab)
```

```
{'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
```

```
pairs = get_stats(vocab)
```

```
[(('es', 't'), 9), (('t', '</w>'), 9), (('l', 'o'), 7), (('o', 'w'), 7), (('n', 'e'), 6)]
```

```
vocab = merge_vocab(pairs[0], vocab)
```

```
{'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
```

Tokeniser

compress frequent patterns

“We tokenize the data with the **bytepair encoding (BPE)** algorithm (Sennrich et al., 2015), using the implementation from SentencePiece (Kudo and Richardson, 2018). Notably, we split all numbers into individual digits, and fallback to bytes to decompose unknown UTF-8 characters.” (—from Llama1 Paper)

- Incrementally combine together the most frequent token pairs

```
{'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
```

```
pairs = get_stats(vocab)
```

```
[(('e', 's'), 9), (('s', 't'), 9), (('t', '</w>'), 9), (('w', 'e'), 8), (('l', 'o'), 7), ...]
```

```
vocab = merge_vocab(pairs[0], vocab)
```

```
{'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
```

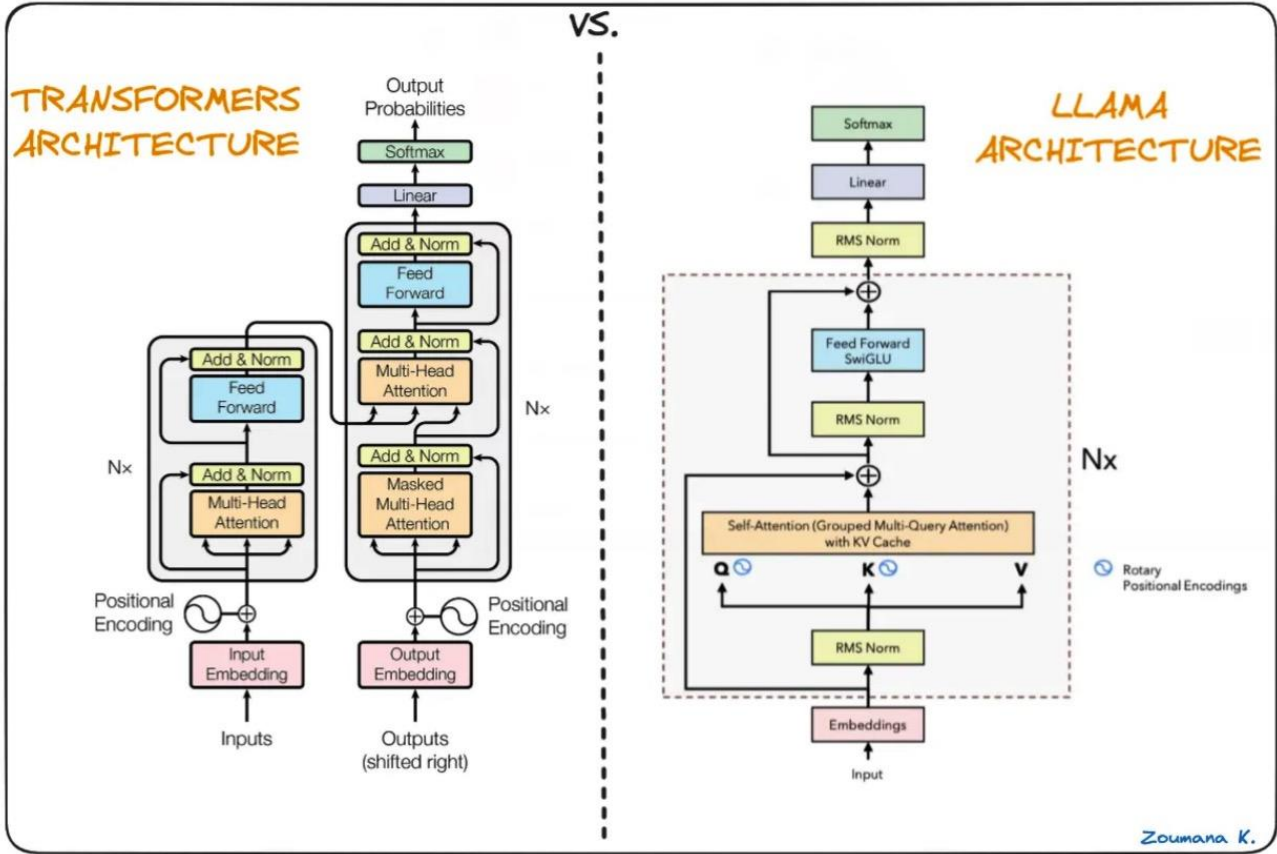
```
pairs = get_stats(vocab)
```

```
[(('es', 't'), 9), (('t', '</w>'), 9), (('l', 'o'), 7), (('o', 'w'), 7), (('n', 'e'), 6)]
```

```
vocab = merge_vocab(pairs[0], vocab)
```

```
{'l o w </w>': 5, 'l o w e r </w>': 2, 'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
```

Architecture



Other Setups

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Table 2: **Model sizes, architectures, and optimization hyper-parameters.**

Optimizer: AdamW (β_1 : 0.9, β_2 : 0.95)

Learning Rate Schedule: Cosine schedule

Final learning rate: 10% of the maximal learning rate

Weight Decay: 0.1

Gradient Clipping: 1.0

Warmup Steps: 2,000 steps

Bigger model $\neq$ better
compute trade-offs

Supervised Fine-tuning

Step 1

**Collect demonstration data,
and train a supervised policy.**

A prompt is
sampled from our
prompt dataset.

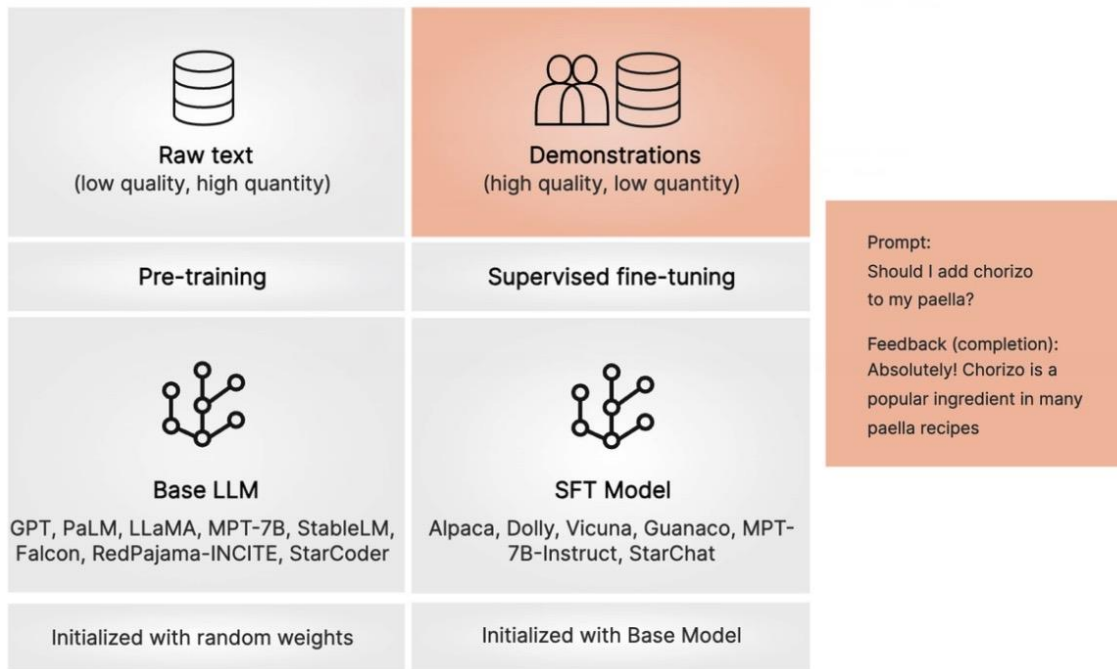
🗣️
Explain the moon
landing to a 6 year old

A labeler
demonstrates the
desired output
behavior.

👤
Some people went
to the moon...

This data is used
to fine-tune GPT-3
with supervised
learning.

SFT
🧠
📄📄📄



SFT adjusts the model's weights to improve its performance on the specific task and relies on high-quality, labeled data for effective results.

RLHF: Reinforcement Learning from Human Feedback

Q: Can RLHF introduce bias?

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



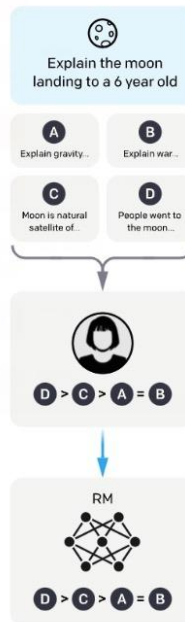
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

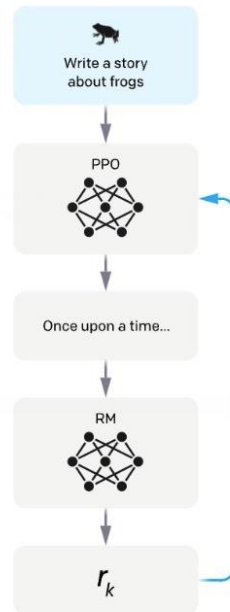
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



RLHF: Reinforcement Learning from Human Feedback

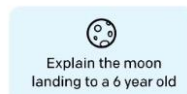
Q: Can RLHF introduce bias?

A: Yes, human values are subjective.

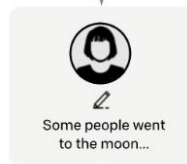
Step 1

Collect demonstration data, and train a supervised policy.

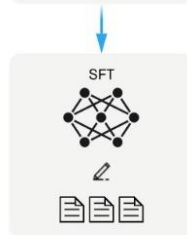
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



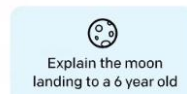
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

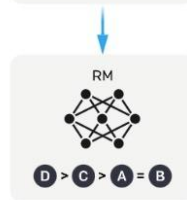
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



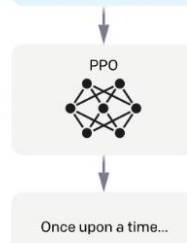
Step 3

Optimize a policy against the reward model using reinforcement learning.

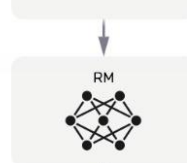
A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Question: What is the difference between inference, generation, and decoding?

LLM Inference

Inference

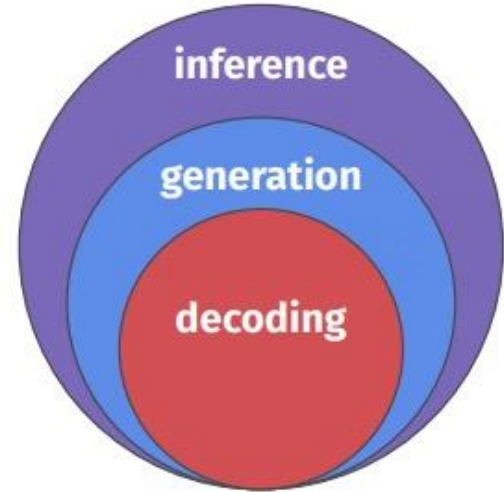
- The concept of using a trained model for making predictions on new data (for classification, sequence tagging, text generation, ...).

Generation

- The process of using a trained model for producing a sequence of tokens.

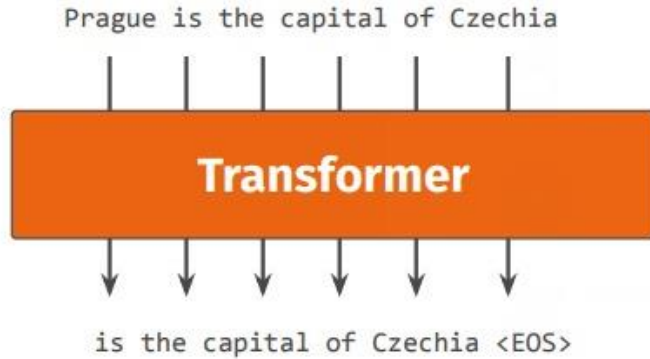
Decoding

- The algorithm of selecting the next token using the model's internal representation.



LLM Inference

Training



Teacher forcing:

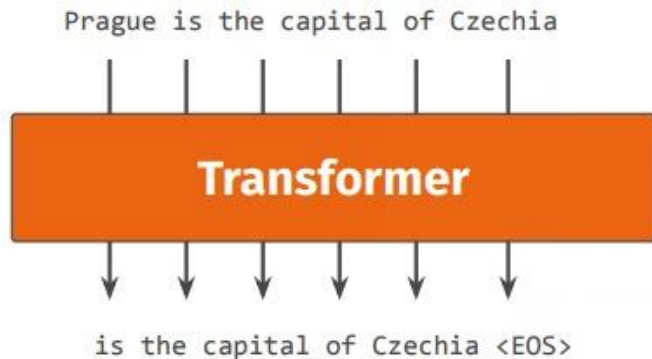
Inference



Decoding: We need to select what token should come next.

LLM Inference

Training



Teacher forcing: We know the previous tokens and what token should come next, so we use them to train the model.

Inference



Decoding: We need to select what token should come next.

Generating text

For every sequence, the LLM generates a probability distribution over the vocabulary of tokens.

To generate text:

1. Start with a sequence of tokens (“prompt”).
2. Feed the sequence into the LLM.
3. Select the next token from the model-generated probability distribution.
4. Append the selected token to the sequence.
5. Repeat from (2).

Autoregressive decoding

$t=1$

<BOS>



I



Transformer



$P(y_t | \text{<BOS>, "I"})$



...



...



...



which token to select?

Autoregressive decoding

$t=1$

<BOS>

I

Transformer

$P(y_t | \text{<BOS>, "I"})$

a

aardwark

am

...

I

...

the

walrus

...

zyzzyva

am

the most probable one?



Autoregressive decoding

$t=2$

<BOS>

I

am

Transformer

$P(y_t | \text{<BOS>, "I", "am"})$

a
aardwark
am

...

I

...

the
walrus

...

zyzzzyva

the

every single time?



Autoregressive decoding

t=3

<BOS>

I

am

the

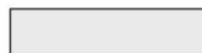
Transformer

$P(y_t | \text{<BOS>, "I", "am", "the"})$



a
aardwark
am

...



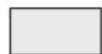
model

...



the
walrus

...



zyzzyva



walrus

YOLO!

Autoregressive decoding

$t=4$

<BOS> →

I →

am →

the →

walrus

Transformer

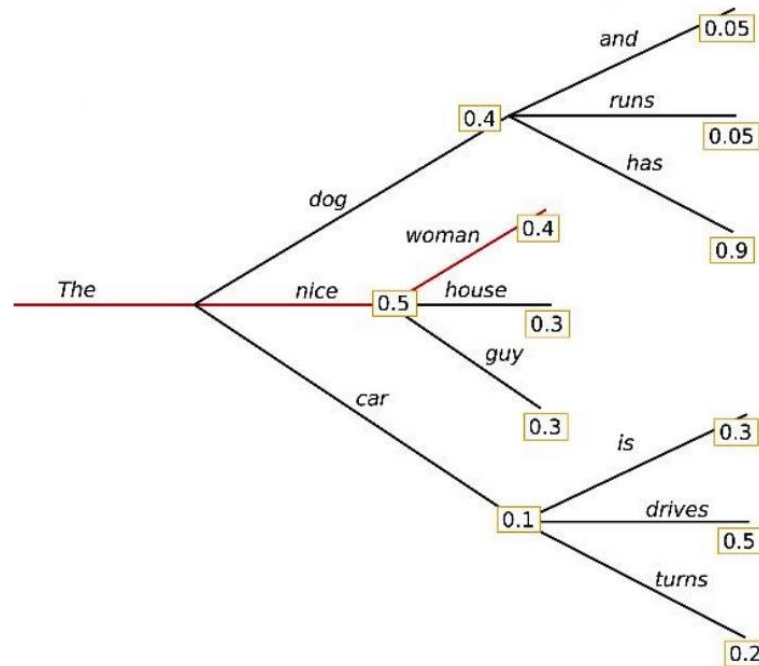


Decoding algorithms - Greedy decoding

- Selecting the **most probable token** in each step t :

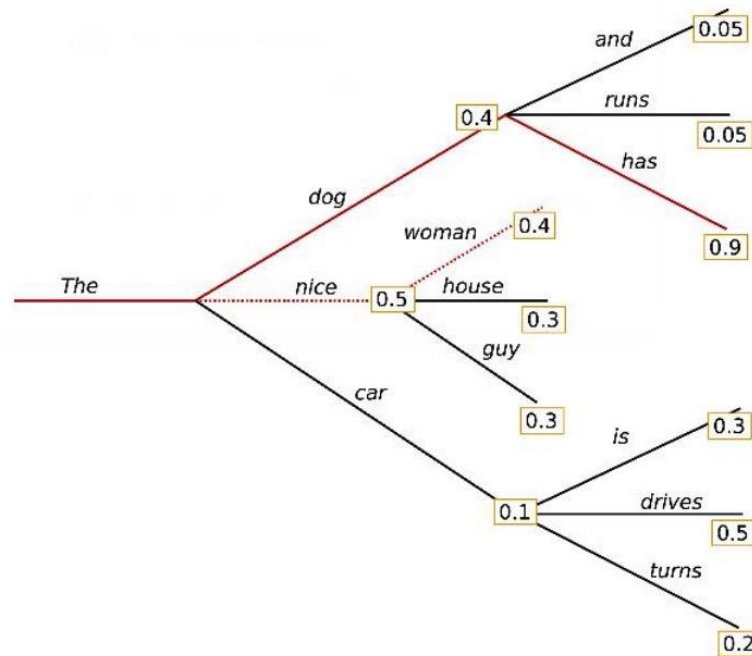
$$y_t = \arg \max_{y_t \in \mathcal{V}} P(y_t | y_1, \dots, y_{t-1})$$

- Very fast, often works satisfactorily (especially with LLMs)
- Non-parameteric



Decoding algorithms - Beam search

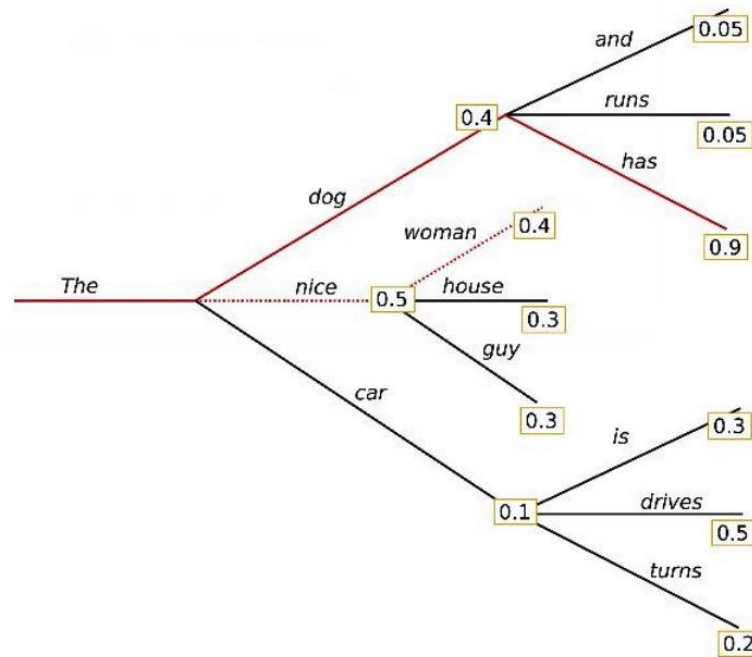
- Parameter **k : number of sequences**
- Each step t :
 - Extend the sequences from the step $t-1$ with all possible tokens.
 - Select the k most probable sequences for the step $t+1$.
- Tuning k :
 - $k=1$ == greedy decoding
 - larger $k \rightarrow$ slower algorithm
 - $k>1$ allows re-ranking results



Trade-off?

Decoding algorithms - Beam search

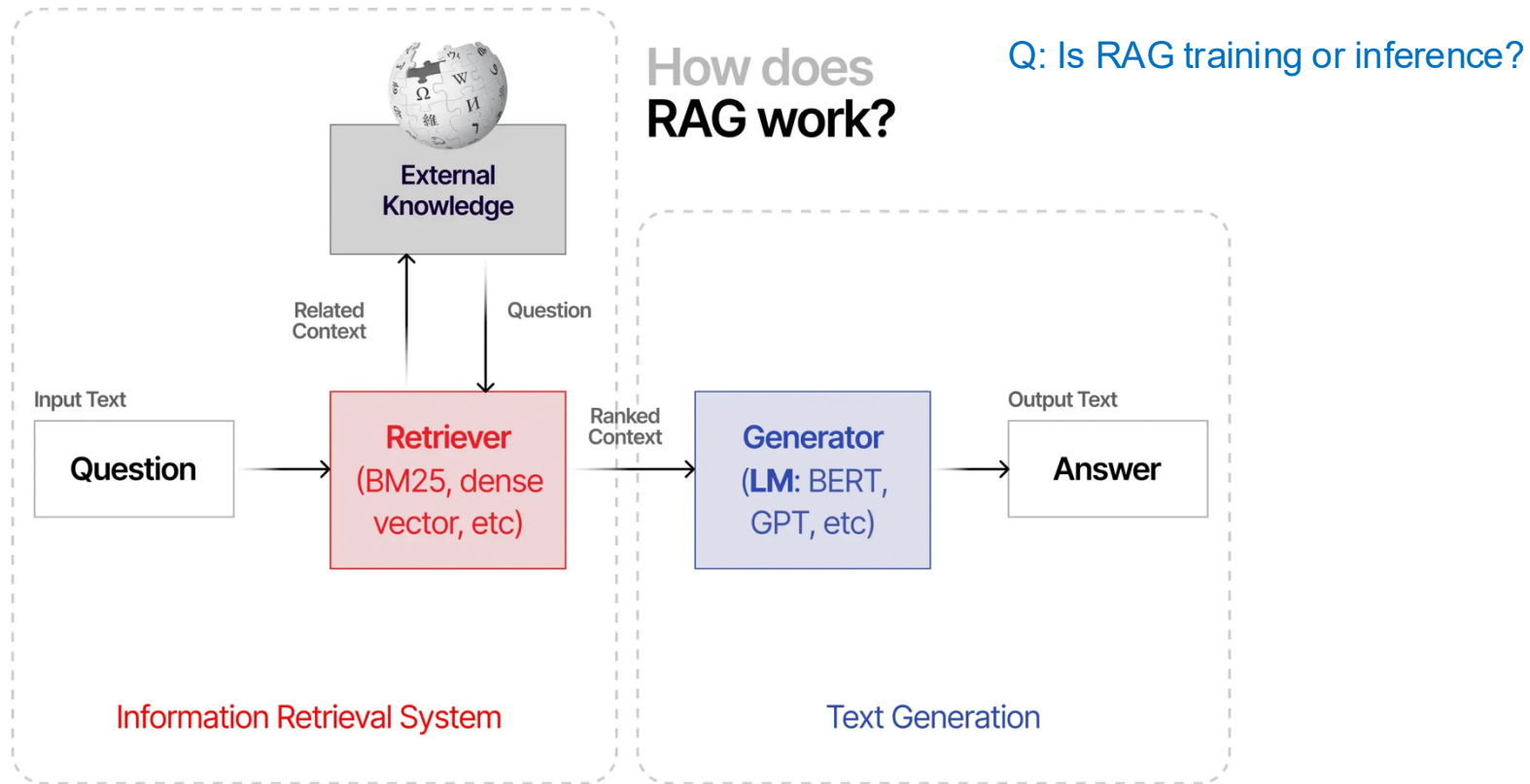
- Parameter **k : number of sequences**
- Each step t :
 - Extend the sequences from the step $t-1$ with all possible tokens.
 - Select the k most probable sequences for the step $t+1$.
- Tuning k :
 - $k=1$ == greedy decoding
 - larger $k \rightarrow$ slower algorithm
 - $k>1$ allows re-ranking results



Trade-off? quality vs speed

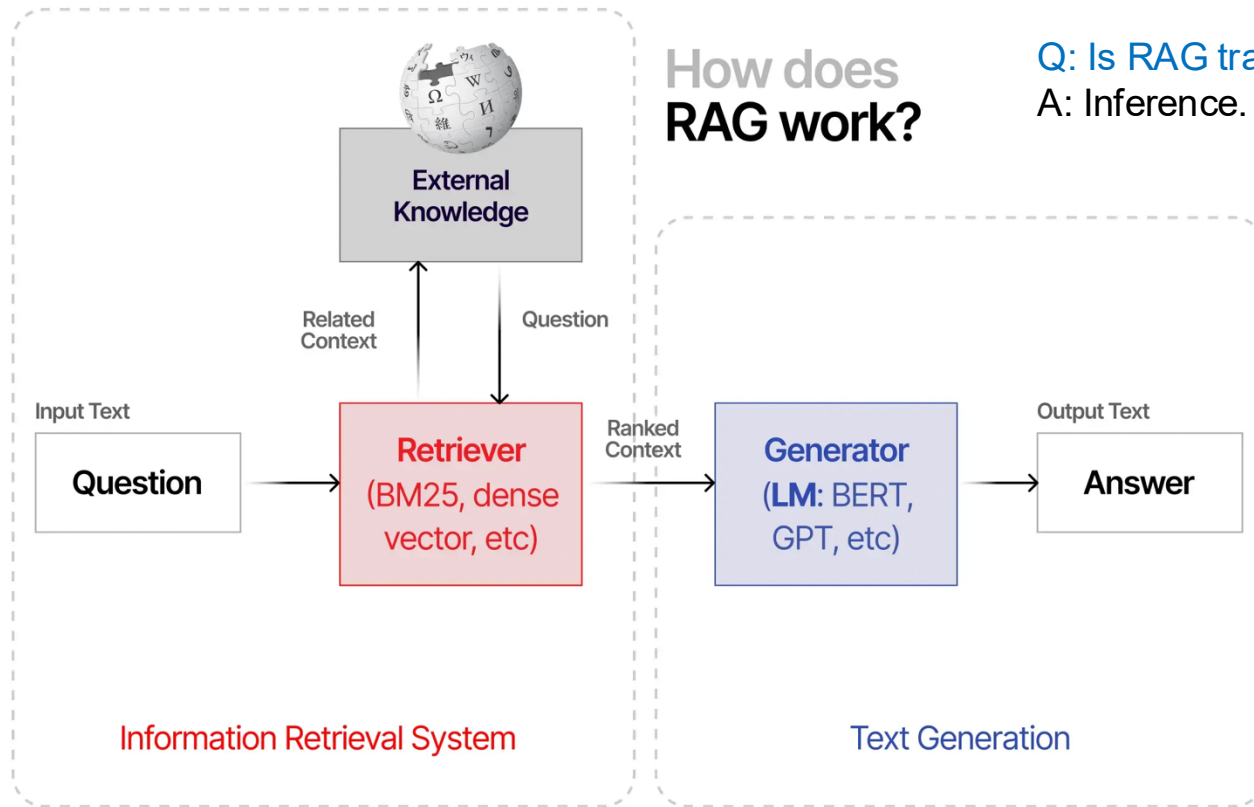
Tricks support prompting/reasoning

RAG: Retrieval-Augmented Generation



Combine a large language model with an external knowledge base to generate more accurate and up-to-date responses.

RAG: Retrieval-Augmented Generation



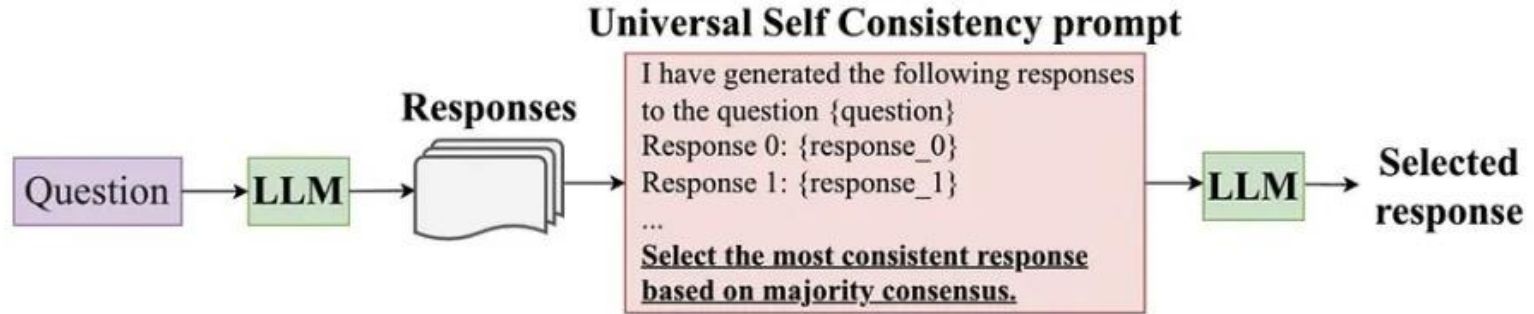
Combine a large language model with an external knowledge base to generate more accurate and up-to-date responses.

Chain-of-Thought (CoT)



Encourages the model to explicitly write intermediate reasoning steps.

Self-Consistency



Generate multiple reasoning paths and select the most common answer.

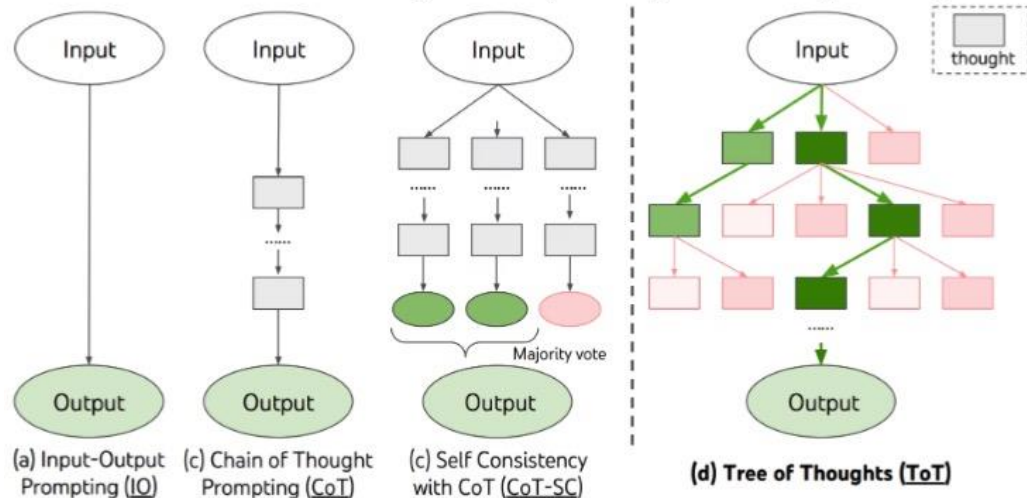
Tree of Thoughts (ToT)

Extends CoT by exploring branching reasoning paths like a search tree.

Process:

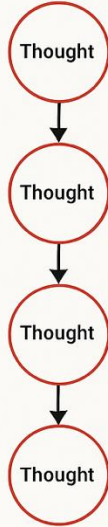
- Generate multiple intermediate states
- Evaluate and expand best ones
- Backtrack if needed

Tree of Thoughts (ToT) Prompting

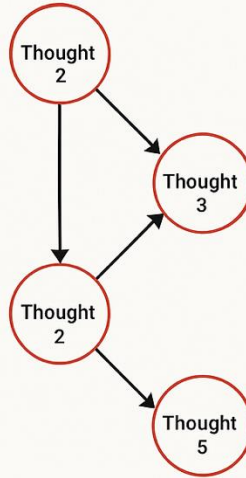


Graph of Thoughts (GoT)

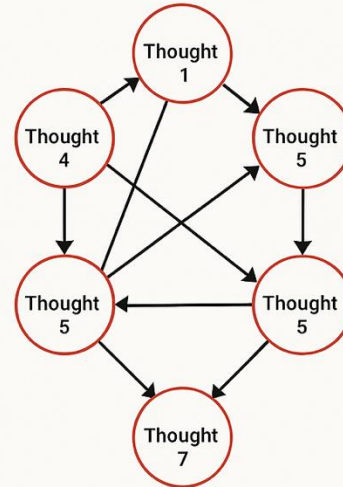
CHAIN-OF-THOUGHT



TREE-OF-THOUGHT



GRAPH-OF-THOUGHT



Generalises ToT by allowing non-linear reasoning and merging paths.

Others

1. Least-to-Most Prompting

Break the problem into simpler subproblems before solving.

2. Program-Aided Language Models (PAL)

Use code to perform reasoning.

LLM generates Python code → code executes → result returned.

3. Debate & Multi-Agent Reasoning

Multiple LLMs argue and critique each other.

4. Reflection / Self-Critique

Model evaluates its own answer.

5. Plan-then-Execute

Two-stage approach:

Generate plan

Execute plan

Prompt Engineering: The “science” to send the exact right message to get the output you want

Why prompting changes behaviour

1. Prompts shift the probability landscape

E.g.,

- Answer briefly: pushes probability mass toward concise completions.
- Explain step-by-step: shifts it toward longer, structured reasoning.

3. Framing effects alter reasoning strategy

Prompt Style	Resulting Behaviour
"Give the final answer"	Direct, often shallow
"Think step-by-step"	More deliberate reasoning
"Explain to a 10-year-old"	Simplified abstraction
"Act as a professor"	Formal, structured discourse

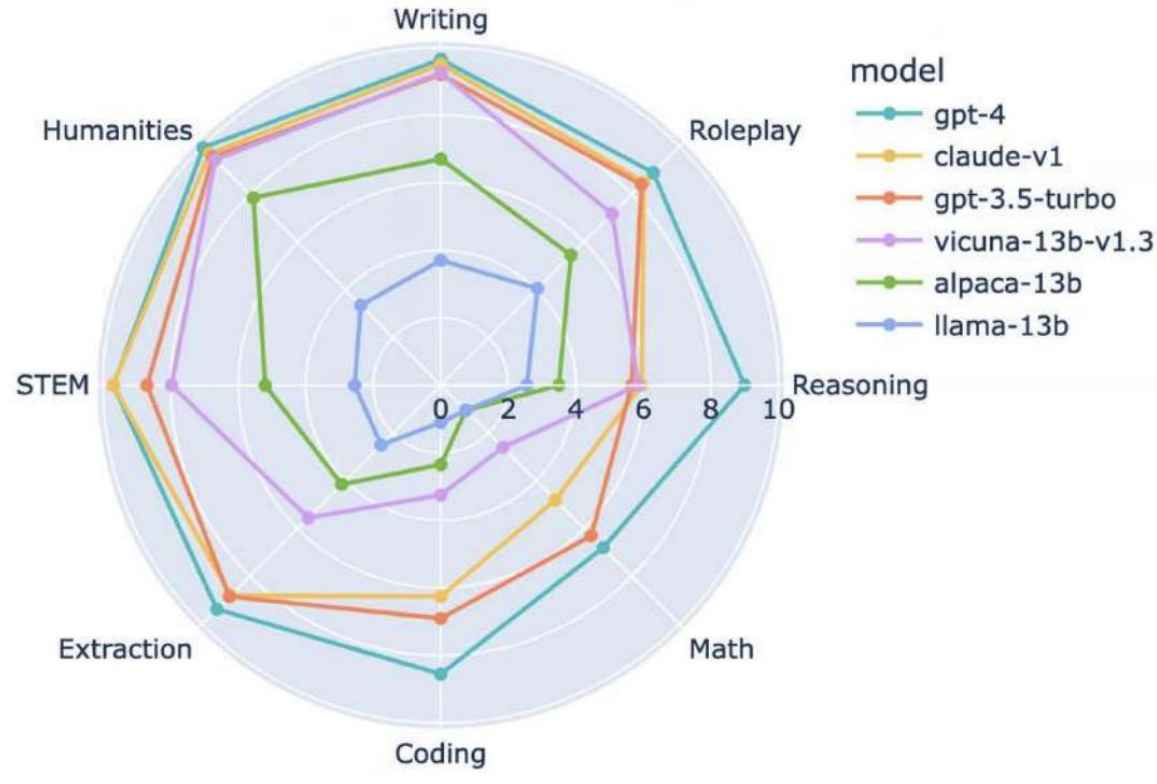
2. Prompts act as implicit task specifications

- Q&A mode
- Tutor mode
- Analyst mode
- Creative writer mode

4. Activation of different internal circuits

Recent interpretability work shows that different prompts activate different neuron subnetworks

LLMs are not good at math



Prompt Engineering Example

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓*

Paper: Large Language Models are Zero-Shot Reasoners

Next: Limitations of LLMs

E.g., Hallucination



How many 'm's are in the word 'Weather'?



There is one 'm' in the word 'Weather'.



Are you sure?



Apologies for the oversight. You're right; there are no 'm's in the word 'Weather'. Thank you for pointing it out.